

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly paired with graph-structured data for node classification and related tasks that have traditionally been evaluated with Graph Neural Networks (GNNs). Although these graph-aware LLMs can exploit relational information to improve task performance, their empirical robustness under adversarial graph and text perturbations has not been quantified systematically. We provide a comparative robustness study across poisoning (training-time) and evasion (test-time) attacks on two representative frozen-LLM architectures, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024), using established graph attacks and matched evaluation conditions. The results show clear, repeatable vulnerability patterns: evasion attacks produce larger accuracy drops than poisoning attacks, LLAGA’s node sequence template exposes a new attack surface in which malicious placeholder nodes sharply reduce accuracy for LLAGA, GRAPH-PROMPTER’s GNN encoder is comparatively more robust to structural perturbations, and both models remain highly sensitive to imperceptible feature perturbations. We further show that a unified structural-plus-feature attack yields the strongest degradation across datasets, while preserving the same underlying perturbation budget and visual imperceptibility constraints. Finally, we propose an end-to-end defense framework, GALGUARD, that combines an LLM-based feature correction module with adapted GNN defenses so that the empirical gains from feature repair and structural purification can be evaluated jointly¹

1 Introduction

Graphs represent entities and relationships through nodes and edges, making them a standard data model for social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and health-care (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018) have therefore served as the dominant empirical baseline for graph tasks such as node classification and link prediction. More recently, large language models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023) have motivated a new comparison point: graph-aware LLMs (Xie et al. 2023), which adapt

LLMs to tasks historically handled by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Prior studies report that these models can outperform GNNs on several graph-related benchmarks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), making their robustness under matched attack settings practically important rather than merely diagnostic.

Despite this growing empirical promise, the adversarial robustness of graph-aware LLMs remains largely unmeasured. Existing black-box attacks on GNNs include poisoning attacks, which perturb training data, and evasion attacks, which perturb inputs only at test time (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023). These attacks evaluate *model integrity* by measuring how much accuracy degrades after subtle changes to graph structure or node features. Because most GNN robustness work emphasizes structural perturbations, the text channel introduced by graph-aware LLMs creates an additional quantitative comparison axis: whether imperceptible feature perturbations can produce accuracy losses that are comparable to, or larger than, classical structural attacks.

Motivated by the shift from GNNs to graph-aware LLMs, the black-box setting used by established graph attacks, and the newly exposed textual feature channel, we ask whether *adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm*. Answering this question requires separating architectural effects from attack effects. Graph-aware LLM performance *highly depends* on how graph information is encoded for the LLM: node features naturally fit the textual interface, whereas graph structure must be converted into relational context through templates, projectors, or graph encoders (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore first categorize existing methods into two encoding families—*textual descriptions*, which express graph information in natural language, and *learned projectors*, which map graph information into the LLM token space. Since ordinary text attacks already apply directly to textual descriptions, our empirical study focuses on learned-projector methods, using two computationally efficient representatives with frozen pre-trained LLMs: LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024).

¹Our code will be released upon acceptance.

Our analysis shows that graph encoding choices produce measurable differences in attack success. In particular, LLAGA’s node sequence template creates a new attack surface: an adversary can inject malicious placeholder nodes and substantially degrade accuracy under the same graph-aware inference pipeline. We also propose an end-to-end graph-aware LLM defense, GALGUARD, that combines an LLM-based feature corrector with adapted GNN-based structural defenses so that feature repair and structure purification are assessed together. To the best of our knowledge, this is the first systematic empirical study of adversarial vulnerability in graph-aware LLMs. Our contributions are:

- We adapt representative GNN adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to compare poisoning and evasion robustness for graph-aware LLMs under consistent datasets, metrics, and perturbation budgets.
- We develop a graph-encoding taxonomy for graph-aware LLMs and use it to explain why different projector designs expose different empirical robustness patterns.
- We identify and evaluate a node sequence template injection attack for LLAGA, showing that placeholder-based graph manipulation can produce larger evasion degradation than standard structural attacks on several datasets.
- We evaluate imperceptible feature perturbations and a unified structural-plus-feature attack, highlighting that feature attacks can dominate structural attacks for graph-aware LLMs.
- We introduce GALGUARD and report defense accuracy under the strongest structural baseline, showing consistent improvements from combining feature correction with structural purification.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs fall into two broad categories that lead to different robustness questions: *textual descriptions of graphs*, where graph information is written as natural language, and *learned graph projectors*, where graph data are encoded into embeddings or structured token inputs that an LLM can process directly. We use this taxonomy to make the empirical comparisons in later sections explicit: attacks on textual descriptions largely reduce to standard text attacks, whereas learned projectors expose graph-specific structural and feature channels. Figure 5 illustrates both categories; Appendix A.1 provides a fuller taxonomy, and Appendix A.2 reviews related GNN attacks.

Graph Representation as a Textual Description. Textual-description methods convert graph data into natural language and query LLMs with graph-specific instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Peruzzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). Multimodal variants link molecule graphs with text (Liu et al. 2023; Su et al. 2022), while prompt-based studies examine how graph structure can be included in LLM

prompts. For example, Huang et al. (2023a) showed that LLMs often process graph prompts as contextual paragraphs, reinforcing the importance of prompt design in work on heterogeneous, dynamic, and large-scale graph tasks (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Other methods use LLMs to improve node features, graph structures, predictors, or feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs, and Duan et al. (2023) introduced an effective textual graph learning approach. These methods provide a direct route to LLM-based graph reasoning, but their scalability and structural fidelity depend heavily on prompt length and prompt engineering.

Graph Representation as a Learned Projector. Learned-projector methods address the limitations of textual descriptions by mapping graph data into embeddings or graph vectors before LLM processing. This family includes LLAGA, which uses node-level templates and a linear projector (Chen et al. 2024), and methods that incorporate GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Other projector designs use graph neural prompts for knowledge graphs (Tian et al. 2024), prompt-based node feature extractors with GNN adapters (Huang et al. 2023b), or fixed-length graph prefixes built with graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). Because these methods preserve distinct graph and text channels, they allow controlled comparisons among structural perturbations, feature perturbations, and combined attacks.

This Work: We focus on projector methods with *frozen pre-trained LLMs* because they reduce computational cost while preserving the graph-to-token interface that is central to our robustness comparisons (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Our two representatives isolate different encoding choices: LLAGA (Chen et al. 2024) uses node-level templates with a linear projector, whereas GRAPHPROMPTER (Liu et al. 2024) uses a GNN with a linear projector. We exclude textual-description methods because they are less scalable for detailed graph structure and do not introduce the same graph-specific attack surfaces evaluated in our experiments.

3 Threat Model

We adopt the standard adversarial graph setting and extend it to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The attacker is black-box with respect to the target model and can manipulate only a limited subset of graph structure or node features, matching practical settings in which adversaries control a small number of bot accounts in a social network or create sparsely linked hoax pages in a Wikipedia-like graph. The quantitative objective is to reduce node-classification accuracy while keeping perturbations small enough to remain plausible under the benchmark constraints.

Model Access. The attacker operates without access to the target architecture or parameters, so each comparison

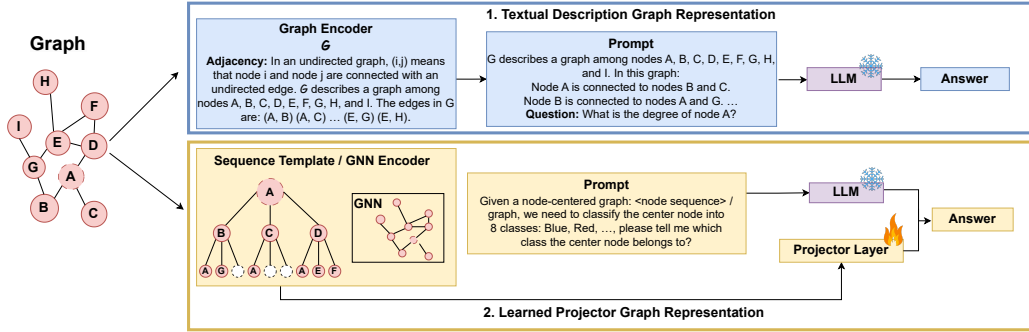


Figure 1: **Graph encoding-based adaptations of LLMs.** The figure separates two comparison groups used in this paper: textual-description methods, which express graph information as prompts (top), and learned-projector methods, which encode graph information through node templates or GNN encoders before mapping it to LLM tokens (bottom). In both projector cases considered here, the LLM remains frozen and only the graph-to-token projector is trained.

measures the effect of data-level perturbations rather than white-box optimization against a specific graph-aware LLM. The adversary can influence the model only through the training or test inputs used in the reported evaluation condition.

Adversarial Capabilities. Consistent with existing poisoning (evasion) attacks, the attacker has access to the training (inference) data of the target model. All modifications remain within a predefined budget and under unnoticeability constraints, which keeps the empirical comparisons tied to the same perturbation scale used by prior graph-attack benchmarks.

Attack Strategies. We evaluate two primary perturbation channels: feature manipulation, which alters node text, and structure manipulation, which adds or removes edges. Reporting results by channel makes the accuracy margins attributable to the attacked input component.

Attacker’s Goal. The attacker’s goal is to compromise model integrity by degrading overall classification performance, and we quantify that goal through the resulting increase in misclassification relative to the corresponding clean or baseline condition.

We evaluate two timing conditions. In poisoning attacks, the adversary perturbs the graph before training, so the projector or graph encoder is trained on corrupted inputs. In evasion attacks, the model is trained on clean data and the adversary perturbs the input only at inference time. This separation lets us compare whether each graph-aware LLM is more sensitive to corrupted training signals or to test-time distribution shifts.

We consider two perturbation channels. Structural attacks modify edges under the same unnoticeability constraint used in prior GNN work, and feature attacks modify textual node attributes through visually imperceptible character-level transformations. The attacker does not change labels, datasets, train/validation/test splits, model architectures, or evaluation metrics; all reported comparisons therefore measure accuracy changes caused by the specified perturbation type, attack method, dataset, and evaluation condition.

4 Experimental Setup

Dataset. We evaluate on three standard text-attributed graph benchmarks—Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016)—so that attack effects can be compared across different graph sizes and sparsity levels. These datasets are widely used for the graph-aware LLM models studied here (Chen et al. 2023a, 2024; Liu et al. 2024). To test whether the same empirical patterns extend beyond small and medium graphs, we also report an additional large-scale ArXiv experiment (Hu et al. 2020) in Appendix A.5. We use the standard 6:2:2 train/validation/test split for Cora, Citeseer, and PubMed, and the 5:2:3 split for ArXiv, following (Chen et al. 2024). Table 1 lists the node counts, edge counts, and sparsity values used to contextualize the robustness comparisons.

Table 1: Dataset statistics for the benchmark graphs used in the main experiments; the node, edge, and sparsity columns define the scale and density conditions for subsequent accuracy comparisons.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We use *node classification* as the common task for all models and attacks, matching the primary evaluation setting in prior graph adversarial work (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023). This keeps the method comparisons aligned around a single accuracy outcome.

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted node labels, for every clean, poisoning, evasion, and defense condition. Each experiment is repeated three times, and tables report the mean and standard deviation so that performance margins can be compared consistently. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Table 2: Accuracy of LLAGA and GRAPHPROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed. Parentheses report the percentage drop from the clean accuracy for the same dataset and model, enabling direct poisoning-versus-evasion and method-versus-method comparisons.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

Choice of Graph-aware LLM and Adversarial Attack Methods. We select graph-aware LLM methods and attack methods to make the main comparisons explicit: architecture-level differences are evaluated through LLAGA and GRAPHPROMPTER, while attack-timing differences are evaluated through poisoning and evasion variants of NETTACK and METAATTACK.

Graph-Aware LLM Methods. We compare two representative frozen-LLM projector methods: LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). Both keep the pre-trained LLM fixed, making the robustness differences attributable to the graph encoding and projector components rather than full LLM retraining. LLAGA uses node-level templates with a linear projector, whereas GRAPHPROMPTER uses a GNN-based encoder followed by a linear projector; in both methods, the projector is a 2-layer multi-layer perceptron (MLP). The foundational LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We evaluate poisoning and evasion variants of two representative graph attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). The first attack is originally a targeted method that searches for minimal perturbations causing selected nodes to be misclassified, whereas the second is untargeted and optimizes graph structure as a hyperparameter. To obtain an untargeted NETTACK comparison, we follow (Li et al. 2023; Zügner and Günnemann 2019) by sequentially targeting nodes and aggregating the resulting accuracy. Across experiments, only 10% of edges are perturbed, preserving the unnoticeability constraint used in (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Appendix A.2 gives the full attack details.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We compare poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to each graph-aware LLM. Under poisoning, LLAGA generates node-level sequences from the perturbed graph and then retraining the projector; GRAPHPROMPTER re-encodes the perturbed graph

with a GAT and retraining its projector. Under evasion, both models are trained on clean graphs, and the same attacks are applied only at inference time, isolating test-time robustness from training-time corruption.

Results. Table 2 shows a consistent timing effect: poisoning attacks produce comparatively small drops, with the largest reported degradation equal to 11%, whereas evasion attacks produce much larger drops, reaching 51%. This margin indicates that the frozen-LLM/projector systems absorb some corrupted training signal but remain sensitive when perturbations are introduced directly at inference. Across attack methods, METAATTACK generally yields lower accuracy than NETTACK under both poisoning and evasion, consistent with its hyperparameter-based optimization of graph edges. Dataset density also shapes the empirical pattern: for LLAGA under METAATTACK evasion, Cora drops by 51%, while the sparser PubMed condition drops by 19%.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

The model comparison in Table 2 shows that LLAGA is typically more susceptible than the GNN-projector model, especially under evasion, because its representation is more directly tied to perturbed structure. LLAGA incorporates graph structure through fixed-length node-level sequences and graph Laplacian embeddings, so each sequence position corresponds to a relative structural role in the original graph. When attacks alter the graph, the node embeddings and the structural positions delivered to the LLM change together, amplifying the accuracy loss. By contrast, the GNN-projector model keeps textual node embeddings unchanged during structural perturbations, and only its GNN component receives the modified graph. This separation lets clean textual features partially counterbalance perturbed structure, explaining the smaller drops observed for that method.

6 Node Sequence Template Injection Attacks

We next evaluate a new template-specific attack surface created by the neighborhood detail template used to construct node sequences. Figure 2 illustrates the attack, and Appendix A.4 provides the algorithm. For a target node u , LLAGA’s neighborhood detail template constructs a *fixed-shape* computational tree whose children are sampled 1-hop neighbors,

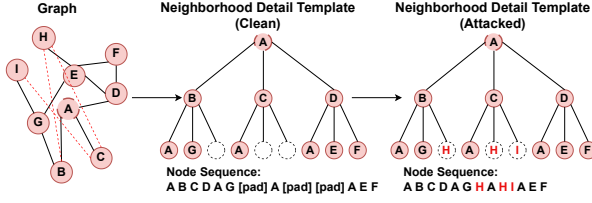


Figure 2: Illustration of the node sequence template injection attacks. The displayed example is Non-adjacent Injection (NI), where red dashed lines mark attacker-added edge perturbations. Supernode Injection (SI) and Multiple Supernode Injection (MSI) use the same placeholder mechanism but insert one or multiple supernodes into the fixed-size node sequence; here, $k = 3$.

repeating the process at subsequent tree levels. When a node has fewer than the required number of neighbors, the template inserts placeholders to preserve a fixed size such as $k = 10$ children, matching LLAGA’s original design (see “Clean” in Figure 2 and lines 1-14 in the algorithm). This fixed-size requirement makes placeholder positions a measurable vulnerability rather than an implementation detail.

We instantiate this attack surface with three increasingly disruptive injection strategies. *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of a target node u (see “Attacked” in Figure 2 and lines 21-27). *Supernode Injection (SI)* inserts a high-degree node as a placeholder while keeping it outside the target node’s 2-hop neighborhood. *Multiple Supernode Injection (MSI)* extends *SI* by inserting multiple distinct supernodes as placeholders (lines 35-43). In all three cases, the quantitative effect is measured through the same accuracy metric: uninformative or misleading nodes enter the sequence, corrupt the target-node representation, and degrade the learned projector at training time or the input representation at inference time.

Results. As shown in Table 3, poisoning remains comparatively mild on Cora and Citeseer, while evasion exposes the largest accuracy margins. On Cora, evasion accuracy decreases to 0.42, 0.36, and 0.30 under *NI*, *SI*, and *MSI*; on Citeseer, the corresponding accuracies are 0.36, 0.30, and 0.24. The monotonic pattern from *NI* to *SI* to *MSI* indicates that stronger placeholder manipulation consistently increases attack effectiveness on these datasets.

On the larger PubMed dataset, the same injection strategies yield smaller evasion losses, with accuracies of 0.85, 0.82, and 0.76. This dataset-level contrast suggests that localized placeholder injections have a more concentrated practical effect in smaller graphs such as Cora and Citeseer, whereas the larger PubMed graph dilutes the same perturbation pattern and preserves more of the neighborhood signal.

Relative to standard structural attacks, the injection attack is often more damaging under evasion. For *MSI*, evasion accuracies are 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, compared with *METAATTACK* accuracies of 0.44, 0.50, and 0.73 in Table 2. Thus, *MSI* improves the attack margin on Cora and CiteSeer while remaining comparable on PubMed.

Table 3: Accuracy under node sequence template injection attacks for LLAGA. Clean is the no-attack baseline for each dataset; *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection, respectively, allowing direct comparison of poisoning and evasion degradation across injection strength.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

7 Poisoning and Evasion Attack via Feature Perturbation

Most graph adversarial attacks evaluate structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019); *NETTACK* (Zügner, Akbarnejad, and Günnemann 2018) also perturbs features, but its feature mechanism does not transfer directly to textual attributes. Because *GRAPH-PROMPTER* remains comparatively robust to structural attacks and because textual features help explain that robustness, we next evaluate whether visually unnoticeable feature perturbations produce larger accuracy drops.

We adapt the imperceptible attack strategies of (Boucher et al. 2022) with one task-agnostic modification: perturbations are generated using an English-to-French translation objective. The perturbation is visually identical or nearly identical to the original text but changes the translation output, indicating that the encoded semantics have been corrupted. We compare two feature attacks: *Homoglyphs*, which replace characters with visually similar alternatives, and *Reorderings*, which use direction-control characters to alter rendering order. Appendix A.6 provides examples of both attacks.

Following (Boucher et al. 2022), we optimize perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between outputs on perturbed and unperturbed text. The feature perturbation budget is fixed at 10% of the average textual-feature length, keeping the feature-attack comparisons aligned across datasets and models.

Results. Table 4 shows that *Reordering* is generally the stronger feature attack across models and datasets. For LLAGA on Cora and CiteSeer, *Homoglyph* poisoning produces drops of 27% and 23%, while *Reordering* poisoning produces larger drops of 35% and 25%, respectively. *GRAPH-PROMPTER* is usually more resilient under the same conditions, but it still loses substantial accuracy under *Reordering*. PubMed follows the same ordering between attack types, although the margins are smaller than on the smaller datasets.

Comparing the feature-attack table with the structural re-

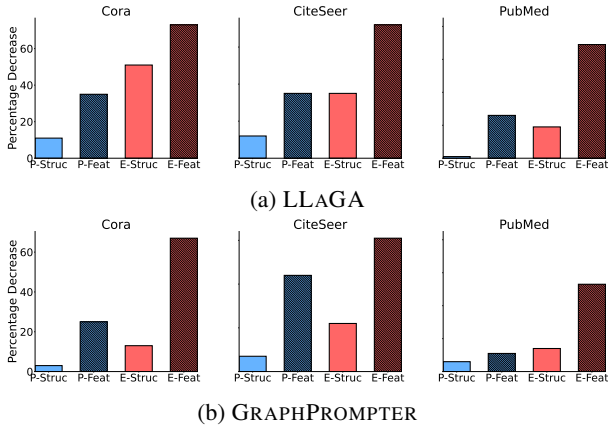


Figure 3: Percentage accuracy decrease for structural and feature-based attacks on graph-aware LLMs. P-Struc and E-Struc denote poisoning and evasion with structural perturbations, while P-Feat and E-Feat denote poisoning and evasion with feature perturbations, making the margin between perturbation channels directly visible.

sults in Figure 3 reveals a central empirical shift: Reordering feature attacks often exceed METAATTACK structural attacks by a large margin for both LLAGA and GRAPH PROMPTER. This pattern indicates that graph-aware LLMs rely heavily on textual node features, so visually subtle feature corruption can be more practically significant than edge perturbation alone. The next section therefore evaluates whether combining the two perturbation channels further increases attack strength.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We next combine feature and structural perturbations into a unified attack and evaluate whether the joint setting produces larger accuracy drops than either channel alone. The comparison preserves the same goal of imperceptibility: structural changes remain subtle, and feature changes remain visually difficult to detect.

For the structural component, we use METAATTACK to adjust edges; for the feature component, we combine Homoglyph and Reordering perturbations as described above. This paired design attacks both sources of evidence used by graph-aware LLMs—relational structure and textual node attributes—under a single evaluation condition.

Results. Table 5 shows that the unified attack yields the strongest degradation across the reported datasets and models. Accuracy drops range from 53% to 69% under poisoning and from 79% to 87% under evasion, which is larger than the corresponding single-channel patterns. The result clarifies the practical significance of the feature findings: graph-aware LLMs are not only affected by structural perturbations, as in traditional GNN settings, but also lose substantial accuracy when textual features are corrupted, and the two channels compound under a joint attack.

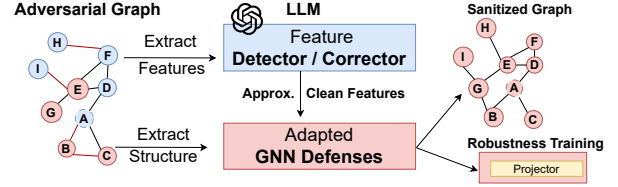


Figure 4: **Overview of GALGUARD defense.** Perturbed node features and edges are colored in blue and red, respectively, separating the two attack channels evaluated in the experiments. The LLM corrector repairs adversarial node features, and the refined features are paired with perturbed edges before applying GNN defenses to obtain a sanitized graph or to support projector robustness training.

9 Ablation Study

9.1 Varying Feature Perturbation

To quantify how attack strength scales with feature corruption, we vary the percentage of perturbed features. Reordering, which follows the same qualitative pattern as Homoglyph, shows a consistent dose-response trend from 1% to 10%: lower perturbation rates produce smaller accuracy drops across LLAGA, GRAPH PROMPTER, Cora, CiteSeer, and PubMed. As the perturbation rate decreases, poisoning and evasion success rates also move closer together. This convergence indicates that when the adversarial signal is weak, the timing of the perturbation matters less because neither training-time nor inference-time corruption is strong enough to dominate the model input. Appendix Figure 2 provides the detailed curves.

9.2 Does the Choice of LLM Affect Attack Performance?

We evaluate whether the foundation LLM changes the robustness conclusions. The default LLAGA backbone is Vicuna-7B (Chiang et al. 2023), while GRAPH PROMPTER uses Llama2-7B (Touvron et al. 2023); we replace LLAGA’s backbone with Llama2-7B and compare poisoning and evasion accuracy across datasets. Clean accuracy remains closely matched (for example, Vicuna: 0.89 ± 0.07 and Llama2: 0.90 ± 0.03 on Cora), and attack performance remains comparable. The same pattern holds when swapping LLMs in GRAPH PROMPTER. These results indicate that, within the evaluated backbones, adversarial robustness is driven more by graph encoding and perturbation channel than by the particular frozen foundation LLM. Table 2 in the Appendix reports the full comparison.

10 Defense: GALGUARD

Existing GNN defenses do not directly match the empirical vulnerability profile observed for graph-aware LLMs because these models are attacked through both structure and text. Many GNN defenses prune or reweight edges using node-feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), which assumes that the features used to compute similarity

Table 4: Accuracy under imperceptible feature perturbation attacks. Homoglyph and Reordering results are reported for poisoning and evasion, with the percentage drop from the corresponding clean accuracy in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

are clean. Once feature perturbations are introduced, this assumption fails (see Figure 3 in the Appendix), so the purified defense variant GALGUARD_p becomes an important intermediate comparison. Similarly, defenses that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021) and OCR-based sanitization (Boucher et al. 2022) do not cover the coordinated structural-plus-feature setting. The attack results therefore motivate an integrated defense whose components correspond to the two measured perturbation channels.

To address this gap, we propose graph-aware LLM defense (GALGUARD), an end-to-end defense strategy combining two complementary approaches: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As shown in Figure 4, the first defense component targets the feature-attack channel by using an LLM to inspect textual node features for anomalous character patterns. The LLM produces approximate corrections that restore feature integrity before graph encoding, directly addressing the feature perturbations that caused large accuracy drops in the attack experiments. The specific instructions and prompts are in Appendix A.6. For all defense experiments, we use GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. The second defense component targets structural perturbations with a two-step procedure. First, we apply a *graph purification heuristic* (Wu et al. 2019) as preprocessing: for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and each neighbor embedding $\phi(x_u)$, then remove low-similarity edges. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). For GRAPHPROMPTER, GNNGuard is applied directly to the message-passing layers. For LLAGA, we adapt the purification step to the computational tree used for node-sequence construction and add a learnable *global structural context embedding* (M_{global}) to the projector. Concatenating M_{global} with the purified node sequence embedding allows training to account for perturbed graph inputs. Because GALGUARD is modular, other structural defenses, e.g. Pro-GNN (Jin et al. 2020), can replace GNNGuard.

10.1 Results

Table 6 reports accuracy under METAATTACK and compares two defense variants: GALGUARD_p, which combines feature

Table 5: Accuracy under the unified imperceptible attack that combines structural and feature perturbations. Parentheses report the percentage drop from clean accuracy for the same dataset and model.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

correction with graph purification, and full GALGUARD, which adds GNNGuard or its LLAGA-specific adaptation. This layout keeps the baseline attack, partial defense, and full defense aligned for each dataset, model, and attack timing condition. Across Cora, CiteSeer, and PubMed, and under both poisoning and evasion, the defense variants consistently improve accuracy relative to METAATTACK. GALGUARD_p already recovers accuracy by pairing feature correction with initial graph purification, while the full defense delivers the strongest value in every reported block. The largest practical gains occur in evasion settings, for example on Cora with the node-template model, where accuracy increases from 0.44 $\pm$ 0.06 under attack to 0.83 $\pm$ 0.04 with GALGUARD.

Table 6: Accuracy under METAATTACK and the proposed defenses for poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification, while GALGUARD adds the adapted GNNGuard component; bold values mark the strongest defense result within each dataset, model, and attack type.

Data	Method	LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper provides a systematic empirical robustness study of graph-aware LLMs under poisoning, evasion, structural, feature, and unified attack conditions. The results show that graph encoding choices materially affect robustness: LLAGA is more sensitive to structural perturbations than the GNN-projector model, and its node sequence template exposes a placeholder-injection attack that produces especially large

evasion drops on Cora and CiteSeer. Across models, imperceptible feature perturbations often exceed structural attacks, and the unified attack produces the strongest degradation, with evasion drops reaching 79%–87% in the reported settings.

We also introduce GALGUARD, an end-to-end defense that aligns with the measured attack channels by combining an LLM-based feature corrector with adapted GNN-based structural defenses. In the structural-attack defense evaluation, the full defense consistently outperforms the attack baseline and the purification-only variant across datasets, models, and attack timing conditions. As graph-aware LLMs are deployed in domains such as social networks, healthcare, and finance, these quantitative patterns emphasize the need to evaluate robustness jointly over graph structure, textual features, and model encoding design.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.

- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejdl, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix provides the supporting comparison details for the main empirical claims. Appendix A.1 expands the graph-aware LLM taxonomy, Appendix A.2 reviews GNN attack baselines, and Appendix A.3 describes how NETTACK and METAATTACK are adapted for the reported poisoning and evasion evaluations. Appendix A.4 gives the node sequence injection algorithm for LLAGA. Appendix A.5 reports ablations on feature-perturbation rate and base LLM choice, Appendix A.7 provides feature-attack examples and the LLM-corrector template, and Appendix A.8 discusses the empirical implications.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has produced multiple adaptation strategies for graph reasoning, representation, and inference. For the robustness analysis in this paper, the most important distinction is how each method encodes graph evidence before it reaches the LLM. We therefore divide existing methods into two comparison groups: *textual descriptions of graphs*, where graph information is encoded as natural language, and *learned graph projector*, where graph data are mapped into embeddings or structured token inputs. Figure 5 illustrates the two groups, and Table 7 summarizes their differences.

Graph Representation as a Textual Description In the first category, graph data are written as natural language so that the LLM receives graph structure through a text prompt. This family includes methods that convert graph data into textual formats and often pair those descriptions with task-specific instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a). For example, Tan et al. (2023) generated random-walk textual sequences for attributed graph embedding, while Zhao et al. (2023) translated graph structures into natural language for graph reasoning in text space. Wei et al. (2024a) augmented textual graph representations with visual graph representations, and multimodal work linked molecule graphs to textual descriptions for drug discovery and molecular retrieval (Liu et al. 2023; Su et al. 2022). Huang et al. (2023a) further showed that LLMs tend to treat graph prompts as contextual paragraphs rather than explicit graph structures, which makes prompt design a major determinant of downstream performance in heterogeneous and large-scale graph settings (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Related work also uses LLM-generated explanations as features to improve graph performance and interpretability (He et al. 2024).

A related textual-description line combines LLMs with GNN pretraining or feature construction. For instance, Chen et al. (2023a) compared using LLMs to enhance node features for node classification with using LLMs as direct graph-task predictors. Chandra et al. (2020) used LLMs as text encoders

and GNNs as graph-structure encoders for fake-news detection, while Chen et al. (2023b) used LLM-generated node annotations to train a GNN without explicit labels. Mavromatis et al. (2023) likewise used an LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a textual-graph fine-tuning method that combines LLMs and GNNs through a tunable side structure at each LLM layer, improving training efficiency and inference speed. Duan et al. (2023) introduced SimTeG, where an LLM is first fine-tuned on a downstream task such as node classification; node embeddings are then extracted from the final hidden states and used for prediction. Zhang (2023) proposed Graph-toolformer, which uses ChatGPT-powered prompt engineering to improve graph reasoning.

These approaches make LLMs accessible for graph tasks, but their empirical performance depends strongly on how much structure can be expressed in the prompt and how well that prompt is engineered. As graphs become larger or structurally more complex, textual descriptions can omit relevant structure or exceed prompt budgets, making scalability and robustness difficult to compare fairly against projector-based methods.

Graph Representation as a Learned Projector Learned-projector methods provide a second comparison group by transforming graph information into embeddings or graph vectors that the LLM can process more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that uses graph tokenization with node-level templates and a linear projector, while Liu et al. (2024) proposed a GNN-based projector. Other work incorporates GNN layers into pre-trained LLMs (Qin et al. 2023). Tian et al. (2024) used graph neural prompts to encode knowledge graphs as graph embeddings for LLM inference. Huang et al. (2023b) introduced G-Prompt, a prompt-based node feature extractor for few-shot learning on text-attributed graphs that combines a learnable GNN adapter with task-specific prompts.

Another line condenses graph information into fixed-length prefixes with graph transformers, attaching them to LLM layers to improve reasoning (Chai et al. 2023). Yang et al. (2021) integrated text encoding and graph aggregation by nesting GNNs alongside transformer blocks, and Peng et al. (2024) represented graphs as sequential paths for natural-language graph interaction.

We do not evaluate projector adaptations that require re-training the LLM because they increase computational cost without being necessary for the robustness comparisons in this paper. Instead, we focus on methods that use **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). These methods train only the projector that maps graph data into tokens, allowing the same frozen LLM interface to be compared under clean, poisoning, evasion, and defense conditions. Throughout this work, we use two representative projector methods as empirical comparison points: **LLAGA** (Chen et al. 2024), which uses node-level templates and a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which uses a GNN-

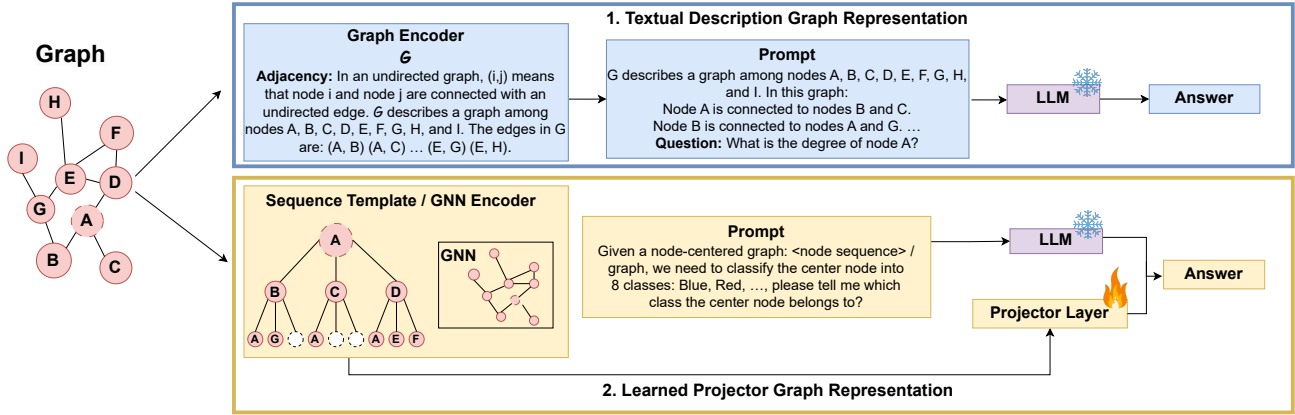


Figure 5: **Graph encoding-based adaptations of LLMs.** The appendix taxonomy compares textual-description encodings (top) with learned-projector encodings (bottom). Within the projector family, node-template and GNN-encoder designs define the architectural comparison used in our robustness experiments; in both cases, the LLM remains frozen and only the graph-to-token projector is trained.

Table 7: Comparison of graph encoding-based LLM adaptations, emphasizing the encoding interface, scalability tradeoff, and attack surface relevant to robustness evaluation.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

based projector to encode graph structure.

This work. We exclude textual-description methods from the main empirical study because their graph structure is constrained by prompt length and often reduces to simpler descriptions such as node degree or edge lists. For larger graphs, the prompt budget can be exhausted before the relevant local or global structure is represented. From a vulnerability perspective, textual-description methods also do not expose the same graph-specific comparison axes. A description such as "Node A is connected to Node B" is primarily vulnerable to standard text attacks, whereas projector methods preserve separate structural and feature channels. Those channels are the focus of our quantitative robustness evaluation.

A.2 Attacks on GNN

GNNs have been evaluated under several attack families: membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), property inference attacks (Zhang et al. 2022; Wang and Wang 2022), adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b). Our experiments focus on adversarial attacks because their outcome is directly comparable to the reported node-classification accuracy: the perturbation succeeds when accuracy decreases under a controlled budget.

Adversarial graph attacks introduce small, carefully crafted *perturbations* to node features or edges with the goal of reducing predictive performance. We use the standard timing distinction: **poisoning attacks** modify training data, whereas **evasion attacks** modify test-time inputs. Zügner, Akbarnejad, and Günnemann (2018) showed that such perturbations can significantly degrade GNN performance through NETTACK, a greedy method that searches for minimal changes that maximize the training loss. Li et al. (2023) analyzed adversarial graph examples from a data-distribution perspective and proposed a defense, while Wu et al. (2019) provided a broad attack-and-defense analysis centered on small graph perturbations. METAATTACK (Zügner and Günnemann 2019) provides the second structural baseline in our comparisons. It performs poisoning and evasion attacks by treating graph structure as an optimizable hyperparameter in a bi-level objective. Meta-gradients quantify how structural changes affect the attacker’s loss after training, enabling edge updates that reduce model accuracy.

Accordingly, we use NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019) as representative attack baselines. Reporting both poisoning and evasion variants lets us compare training-time integrity loss with inference-time robustness loss under the same graph-aware LLM evaluation metric.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs graph structure by adding and removing edges while preserving the degree distribution of the perturbed graph. It greedily searches for minimal perturbations that maximize the training loss of a surrogate lin-

earized 2-layer graph convolutional network (GCN), which approximates how structural changes affect node classification. The attack scores candidate edge changes around a target node and applies the highest-scoring add or remove operation. Because NETTACK is targeted by design, we follow (Zügner and Günnemann 2019) to obtain an untargeted accuracy comparison: test nodes are attacked sequentially, and the node-classification results are aggregated across the evaluated nodes.

METAATTACK optimizes the adjacency matrix directly by treating graph structure as a hyperparameter in a bi-level objective. The inner level minimizes training loss on the original graph, and the outer level optimizes the poisoned graph objective through a surrogate linearized two-layer GCN. A perturbation score derived from meta-gradients identifies which edge additions or removals most reduce downstream accuracy, after which the algorithm greedily applies the highest-impact changes.

For both attack methods, we use the same unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018): the graph degree distribution may change only slightly, and the perturbation budget is fixed at 10% of edges.

In the poisoning setting, LLAGA uses the perturbed graph from NETTACK or METAATTACK to generate node-level sequences before retraining the projector, while GRAPH-PROMPTER re-encodes the perturbed graph with a graph attention network (GAT) and retrains its projector. In the evasion setting, both graph-aware LLMs are trained on clean data and receive perturbed graphs only at inference, making the reported accuracy drops directly attributable to test-time perturbation.

A.4 Algorithm for Node Sequence Template Injection Attacks

We present the algorithm for the node sequence template injection attack in LLAGA in Algorithm 1. The model constructs a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14). When a node has fewer than k neighbors, placeholders are inserted, creating an attack surface. We exploit this by injecting malicious nodes into placeholder positions through graph manipulation. We propose three strategies: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* injects a high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). These uninformative nodes corrupt the node sequence, degrading the projection layer’s performance. The inclusion of uninformative nodes in the sequence corrupts the target’s representation, impairing training of the projection layer.

A.5 Ablation Studies

This section reports the ablation results used to separate perturbation-rate effects from base-LLM effects.

Ablation Results: Varying Feature Perturbation Figure 6 compares LLAGA and GRAPH-PROMPTER as the per-

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow 2\text{-hop neighborhood of } u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:       break ▷ Only use first placeholder
34:     end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

centage of perturbed features changes. Accuracy decreases as the perturbation rate increases, showing a consistent dose-response pattern for feature-level attacks across both graph-

aware LLMs.

Ablation Results: Does the Choice of LLM Affect Attack Performance? Table 8 isolates the base-LLM factor in the LLGA framework. Across Cora, Citeseer, and PubMed, Vicuna-7B and Llama2-7B produce nearly identical clean accuracies and similar degradation under poisoning and evasion. For example, under METAATTACK evasion on Cora, accuracy is 0.44 ± 0.06 with Vicuna-7B and 0.43 ± 0.08 with Llama2-7B, a difference that lies within the reported standard deviations; the same close alignment holds for NETTACK. The same pattern across attack types indicates that the graph encoding and perturbation channel, rather than the frozen base LLM, drive the observed robustness trends.

A.6 Results on the ArXiv Dataset

We also evaluate node sequence template injection on the large-scale ArXiv dataset, which has 169,343 nodes, 1,166,243 edges, and sparsity 0.81. Table 9 shows a different poisoning-versus-evasion pattern from Cora, Citeseer, and PubMed: poisoning produces larger degradation than evasion across all three injection strategies. Clean ArXiv accuracy is 0.73 ± 0.02 ; poisoning reduces it to 0.50 ± 0.03 , 0.43 ± 0.04 , and 0.42 ± 0.03 under NI, SI, and MSI, whereas evasion remains higher at 0.71 ± 0.06 , 0.69 ± 0.02 , and 0.60 ± 0.03 . This contrast suggests that ArXiv’s structure makes training-time corruption more damaging while still showing that supernode insertion can affect performance despite low sparsity.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples in the prompt, as shown in Appendix A.7, illustrate our adversarial attacks on real features from the Cora dataset. The “input” is the text received by the LLM where the attack is being applied. It is important to note that while a human reader perceives the original text and the perturbed text as visually identical or very similar, the LLM processes a sequence of different Unicode characters or a different character order, which can alter its behavior. We show this in Table 10 for homoglyph and Table 11 for reordering attack.

Feature Corrector Defense Template We designed a feature corrector module to mitigate the effects of imperceptible attacks, specifically homoglyph substitutions and character reorderings. This module is implemented as a prompt-based defense using LLMs. The prompt is carefully crafted to instruct the LLM to act as a linguistic validator, focusing on detecting and correcting subtle, human-imperceptible perturbations.

The prompt template below states the correction task and output format explicitly so that the LLM returns a corrected feature string rather than an unconstrained explanation.

A.8 Detailed Discussion

The results identify several empirical patterns that are important for designing graph-aware LLMs. Because these models are increasingly used for graph tasks, robustness should be evaluated alongside clean performance. Our comparison of

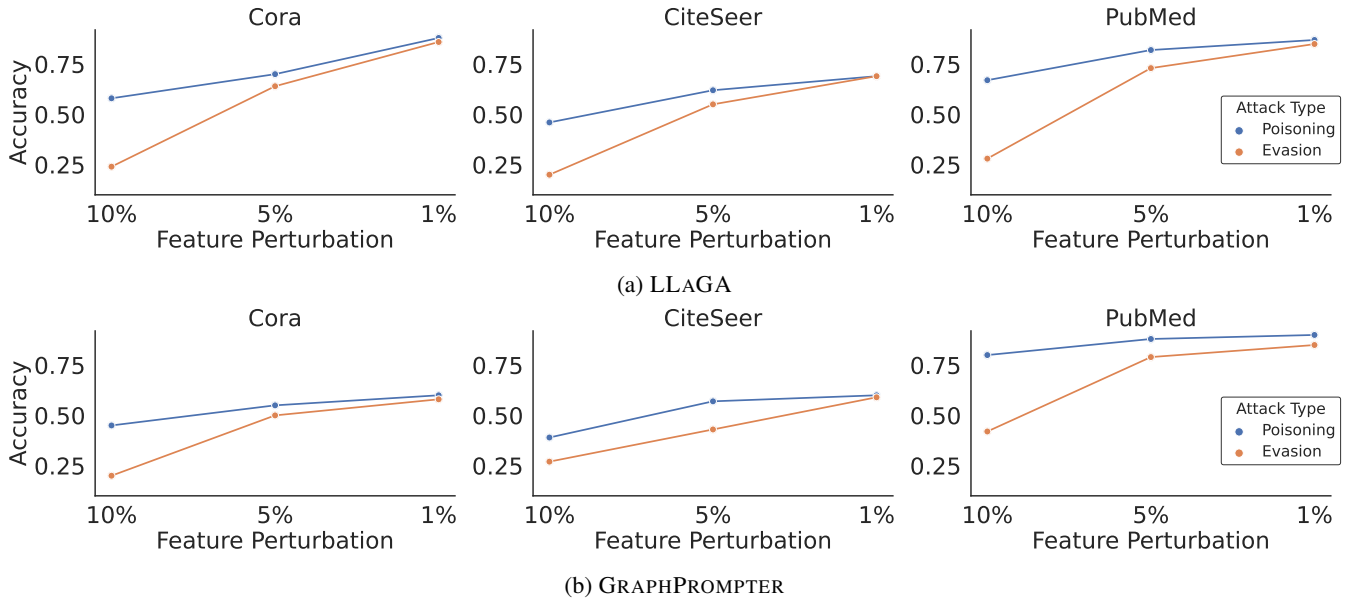


Figure 6: Accuracy of graph-aware LLMs as the feature perturbation percentage varies. Lower accuracy indicates a stronger attack, making the perturbation-rate trend directly comparable for LLAGA and GRAPHPROMPTER.

Table 8: Accuracy with Vicuna-7B and Llama2-7B backbones under clean, poisoning, and evasion conditions. Similar values across rows show that changing the frozen base LLM has limited effect on susceptibility to NETTACK, METAATTACK, or MSI for LLAGA and does not alter the comparison with NETTACK.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

Table 9: Accuracy under node sequence template injection attacks across datasets, including ArXiv. Clean reports the no-attack baseline; *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection, respectively.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	—	0.73 ± 0.02	—
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

LLAGA and GRAPHPROMPTER across poisoning, evasion, structural, feature, and unified attacks shows that vulnerability depends on the graph encoding, the perturbation channel, and the dataset. The main implications are summarized below.

Impact of Graph Encoding on Adversarial Vulnerability The results show that graph encoding can change the attack margin. LLAGA’s node sequence template is effective for incorporating graph structure, but the same fixed-shape sequence mechanism creates the placeholder-injection vulnerability quantified in the main experiments. The attack surface is specific to LLAGA, while GRAPHPROMPTER, which uses a GNN-based projector, is comparatively more resilient to these structural attacks. This contrast suggests a design trade-off: stronger explicit structural encoding can improve graph task performance while exposing new graph-specific attack surfaces.

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The megaprior heuristic fr discvring prtin squnc ptttrns Abstract: Svrl cmputr lgrithms fr discvring ptttrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly allcot th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: Prompt template for the LLM-based feature corrector. The template fixes the role, task, and output format used to repair perturbed textual node features before evaluation.

Table 10: Illustration of the imperceptible homoglyph attack on a Cora feature. The LLM processes distinct Unicode characters, while the rendered text remains nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtin squnc pptrns Abstract: Svrl cmputr lgrithms fr discvring pptrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttistical mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Illustration of the imperceptible reordering attack on a Cora feature. Direction-control characters change the sequence processed by the LLM while preserving a visually similar rendering for humans.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Sensitivity of Graph-aware LLMs to Feature Perturbations Feature perturbations are effective across both graph-aware LLMs, even though the models encode graph structure differently. The Reordering and Homoglyph results show that visually subtle node-feature changes can produce large accuracy drops, making feature robustness a central quantitative finding rather than a secondary concern. Text sanitization methods such as those discussed by (Boucher et al. 2022) may help, but imperceptible homoglyph and reordering attacks remain difficult; the LLM corrector is therefore motivated by the need to repair suspicious textual features before graph-aware inference.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs The poisoning-versus-evasion pattern is dataset dependent. Cora, Citeseer, and PubMed show the main-paper trend in which evasion is often

more damaging, whereas ArXiv shows larger degradation under poisoning for node sequence injection. These differences indicate that robustness should be reported with dataset scale, sparsity, and attack timing rather than as a single aggregate claim.

Implications for Graph-aware LLM Development The empirical patterns suggest that graph-aware LLM development should report robustness alongside clean accuracy. Encoding choices, feature handling, and structural defenses all affect the observed margins, so future systems should be evaluated under both GNN-style structural attacks and LLM-style feature attacks. Understanding when attacks transfer from GNNs to graph-aware LLMs can guide defenses that preserve practical performance while reducing adversarial sensitivity.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** GNN adversarial work has emphasized structural perturbations because feature perturbations often do not change the predicted class of a target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019). As a result, there are *no known defenses against feature-based attacks on GNNs* designed for the feature threat model evaluated here. Our results show that this assumption changes for graph-aware LLMs: feature perturbations can produce large accuracy drops, so defenses must address both structural and textual channels. OCR-style sanitization can help in principle, but imperceptible attacks such as homoglyph substitutions remain challenging because OCR can misinterpret homoglyphs more often than unperturbed text (Boucher et al. 2022). This motivates our LLM-based self-correction mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based attacks transfer from GNNs to graph-aware LLMs, especially under evasion, one might expect GNN defenses to transfer as well. The feature-attack results show why this is insufficient: unlike standard GNN settings, *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective* when the defense assumes clean textual features.

A common GNN defense direction makes the adjacency matrix trainable so edge weights can adapt during training. These methods downweight edges between dissimilar nodes and upweight edges between nodes with higher feature similarity. Related **preprocessing-based** defenses remove low-similarity edges (Wu et al. 2019) or randomly drop edges (Dai et al. 2018). Their effectiveness depends on clean node features for reliable similarity scores; when textual features are perturbed, the similarity graph itself can become misleading. Figure 8 illustrates this failure mode on a random Cora node. The clean feature-similarity graph preserves some original edges while adding some incorrect ones, but the graph reconstructed from perturbed features preserves no original connections, showing how feature corruption can undermine structural preprocessing.

Another defense line reduces reliance on graph structure. For example, (Shanthamallu, Thiagarajan, and Spanias 2021) trained a feature-only surrogate predictor and used uncertainty matching to extract graph information from the GNN, while Wu et al. (2021) trained separate feature and structural submodels in a co-training framework. These approaches still assume unperturbed features, which does not hold in our combined structural-and-feature setting. Consequently, GNN structural defenses are not directly sufficient for graph-aware LLMs; defenses must be tailored to the joint empirical vulnerability profile.

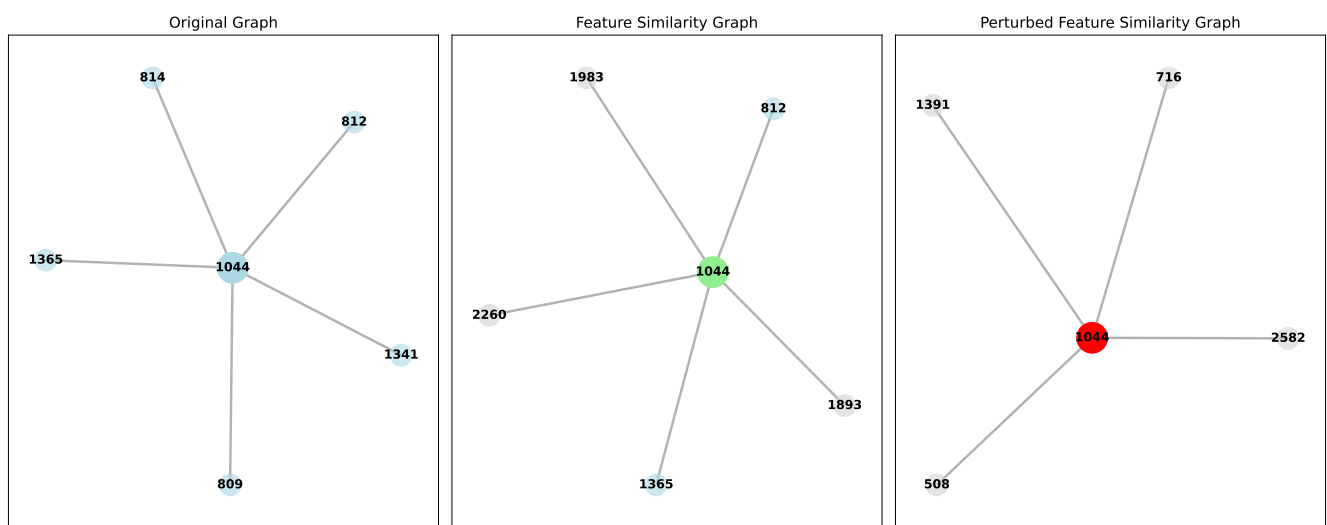


Figure 8: Edges reconstructed for random node 1044. **Original (left)** shows ground-truth graph edges. **Feature Similarity (middle)** reconstructs edges from clean node features, with blue nodes marking preserved original connections. **Perturbed Feature Similarity (right)** reconstructs edges from perturbed features, illustrating the loss of preserved original connections under feature corruption.